
Riemannian Laplace Approximation: Extensions and Improvements

Prem Kansagra
220816

Harsh Agrawal
220425

Arshit Narang
220209

Department of Computer Science,
Indian Institute of Technology, Kanpur
CS772: Probabilistic Machine Learning Project

Abstract

Posterior inference is central to Bayesian machine learning, but exact computation is often infeasible in high-dimensional models, making the use of approximate methods necessary. While traditional Laplace approximations offer a convenient Gaussian approximation of the posterior, they fail to capture the true geometric complexity of modern models. The Riemannian Laplace Approximation (RLA) addresses this limitation by leveraging Riemannian geometry to incorporate local curvature into the approximation. Despite its theoretical strengths, the RLA approach suffers from significant practical challenges—most notably high computational cost due to ODE integration. In this project, we describe the core components of RLA, check its performance on diverse datasets and made improvements to original code to improve its efficiency. An attempt towards implementing a GPU-based ODE solver was also made.

1 Problem Description

Posterior computation lies at the core of Bayesian machine learning. However, exact posterior inference is typically intractable due to the high dimensionality of the parameter space and the difficulty of computing the normalization constant. As a result, approximate inference methods have become essential for practical applications.

One widely used approach is to approximate the posterior with a Gaussian distribution:

$$p(\theta \mid D) \approx q(\theta \mid D) = \mathcal{N}(\theta \mid \mu, \Sigma),$$

estimated via variational methods or Laplace approximation. However, such methods rely on a local quadratic approximation of the log-posterior, which can be severely limiting. Empirical studies, such as [3], have shown that posterior landscapes in modern models (e.g., neural networks) are often highly non-convex with complex curvature, violating assumptions that justify Gaussian approximations.

To address these limitations, [1] introduced the Riemannian Laplace Approximation (RLA). This approach leverages Riemannian geometry to capture the local curvature of the parameter space more accurately. By defining a Riemannian metric informed by the gradient of the loss, RLA enables a more faithful local approximation of the posterior, particularly in curved or anisotropic regions of the parameter space.

While RLA represents a significant conceptual advance, it introduces new challenges that hinder its practical adoption. First, the method is computationally expensive due to the need to integrate geodesic ordinary differential equations (ODEs). Second, the mathematical formulation requires knowledge of advanced differential geometry concepts (e.g., exponential maps, geodesics, normal

coordinates), which may be inaccessible to many practitioners. Third, the reference implementation provided by the authors is not optimized for scalability or efficiency, limiting its utility in large-scale or real-time settings.

This work seeks to address these challenges by analyzing and improving the RLA framework. Our goal is to retain the geometric fidelity of RLA while reducing its computational overhead and simplifying its implementation, thereby making it more practical for a wider range of Bayesian inference tasks.

2 Paper Description

This paper[1] addresses an important problem in Bayesian inference: approximating the posterior distribution of a model’s parameters. Traditional methods, such as the Laplace approximation, assume that the posterior distribution around its mode is approximately Gaussian. However, in many cases, the posterior may not be well-approximated by a Gaussian due to the non-concave nature of the loss function or the complex geometry of the parameter space. The main contribution of this paper is the introduction of the Riemannian Laplace approximation, which adapts the standard Laplace approximation by accounting for the geometry of the parameter space. This is particularly useful when the parameter space Θ has a non-Euclidean structure, such as when the parameters lie on a manifold rather than in flat Euclidean space. The basic idea is to retain the Laplace approximation but change the parameter space Θ to locally encode the training loss. To realize this idea, authors have first endowed the parameter space with a suitable Riemannian metric and then constructed a Laplace approximation according to this metric.

2.1 Key Concepts of the paper:-

- **Parameter Space as a Manifold:** In standard Bayesian inference, we think of the parameters $\theta \in \Theta$ as coordinates in a Euclidean space $\mathbb{R}^K$. However, in this paper, the parameter space Θ is viewed as a manifold, meaning it can have a more complex, curved structure. The loss function $L(\theta)$ is then interpreted as defining a smooth surface or curve in a higher-dimensional space, where the goal is to understand how the posterior distribution behaves on this surface.
- **Riemannian Geometry:** The paper defines a Riemannian metric $M(\theta)$ on the manifold, which essentially tells you how to measure distances and angles on the manifold. This metric is based on the Jacobian of the loss function, which gives the rate of change of the loss with respect to the parameters. This Jacobian provides a natural way to define the tangent space at each point θ , which consists of the directions in which you can move along the manifold. The metric is defined as:

$$M(\theta) = J_g(\theta)^\top J_g(\theta)$$

The Jacobian of g is particularly simple:

$$J_g(\theta) = I_K \nabla_\theta L$$

such that the metric takes the form:

$$M(\theta) = I_K + \nabla_\theta L(\theta) \nabla_\theta L(\theta)^\top$$

- **Exponential Map:** One of the most important concepts introduced in the paper is the *exponential map*, denoted by Exp . The exponential map takes a tangent vector (a direction in which you could move along the manifold) and maps it back onto the manifold itself. Geometrically, this means that if you know a direction from a point θ^* on the manifold, the exponential map tells you where you end up by moving along that direction. In short, the exponential map provides the shortest path between two points on a curved manifold, which is called a *geodesic*. The length of a curve $c(t)$, parameterized over $[0, 1]$, is defined using the Riemannian metric $M(\theta)$ as:

$$\text{length}[c] = \int_0^1 \langle \dot{c}(t), M(c(t)) \dot{c}(t) \rangle dt$$

where $\dot{c}(t) = \frac{d}{dt}c(t)$ is the velocity of the curve, and $M(c(t))$ is the Riemannian metric at each point along the curve. The distance between two points θ_1 and θ_2 on the manifold is the length of the shortest connecting curve, which is given by the geodesic connecting these points.

- **Normal Coordinates:** Normal coordinates are used to simplify the local geometry of a manifold, providing a coordinate system around a reference point (typically the mode of the posterior). Unlike tangential coordinates, which are based on the tangent space and can be influenced by the manifold's curvature, normal coordinates “flatten” the manifold locally. In the manifold, tangent coordinates $\mathbf{v}$ represent a vector in the tangent space at a point $g(\theta)$, and they are related to the coordinates of the tangent space spanned by the Jacobian $J_g(\theta)$.

$$\langle \mathbf{v}, M(\theta)\mathbf{v} \rangle = \langle \bar{\mathbf{v}}, \bar{\mathbf{v}} \rangle$$

where $M(\theta)$ is the Riemannian metric matrix. This equation implies that $\mathbf{v}$ can be transformed into normal coordinates by:

$$\mathbf{v} = A(\theta)\bar{\mathbf{v}}$$

where $A(\theta) = M(\theta)^{-\frac{1}{2}}$ is a matrix that orthonormalizes the basis and simplifies the metric. Normal coordinates make the metric vanish at the reference point, i.e., at θ^* . This leads to a local, flat approximation around the point θ^* , which is useful for approximations like the Laplace approximation.

- **Gaussian Approximation in Normal Coordinates:** To approximate the posterior locally around the mode θ^* , the loss function is reparameterized in *normal coordinates* using the exponential map:

$$h(\bar{v}) = \mathcal{L}(\text{Exp}_{\theta^*}(M(\theta^*)^{-1/2}\bar{v}))$$

This allows a second-order Taylor expansion of the loss around $\bar{v} = 0$:

$$\hat{h}(\bar{v}) \approx h(0) + \langle \nabla_{\bar{v}} h(0), \bar{v} \rangle + \frac{1}{2} \langle \bar{v}, H_{\bar{v}} h(0) \bar{v} \rangle$$

Since θ^* is a minimizer of the loss, the gradient term vanishes, and we obtain a Gaussian approximation $q(\bar{v}) = \mathcal{N}(\bar{v} | 0, \Sigma)$, where

$$\Sigma = (H_{\bar{v}} h(0))^{-1} = (A(\theta^*)^\top H_\theta[\mathcal{L}](\theta^*) A(\theta^*))^{-1}$$

By mapping back to *tangential coordinates* using $v = A(\theta^*)\bar{v}$, we recover the standard Laplace posterior with covariance $H_\theta[\mathcal{L}](\theta^*)^{-1}$, showing that this geometrically grounded formulation yields the same Gaussian approximation while enabling more principled manipulation in curved parameter spaces.

- **Predictive Posterior via Riemannian Laplace Approximation:** In Bayesian inference, the predictive posterior is obtained by marginalizing over the model parameters:

$$p(y | x', D) = \int p(y | x', D, \theta) q(\theta) d\theta$$

Under the Riemannian Laplace framework, we reparameterize this using the exponential map from the tangent space:

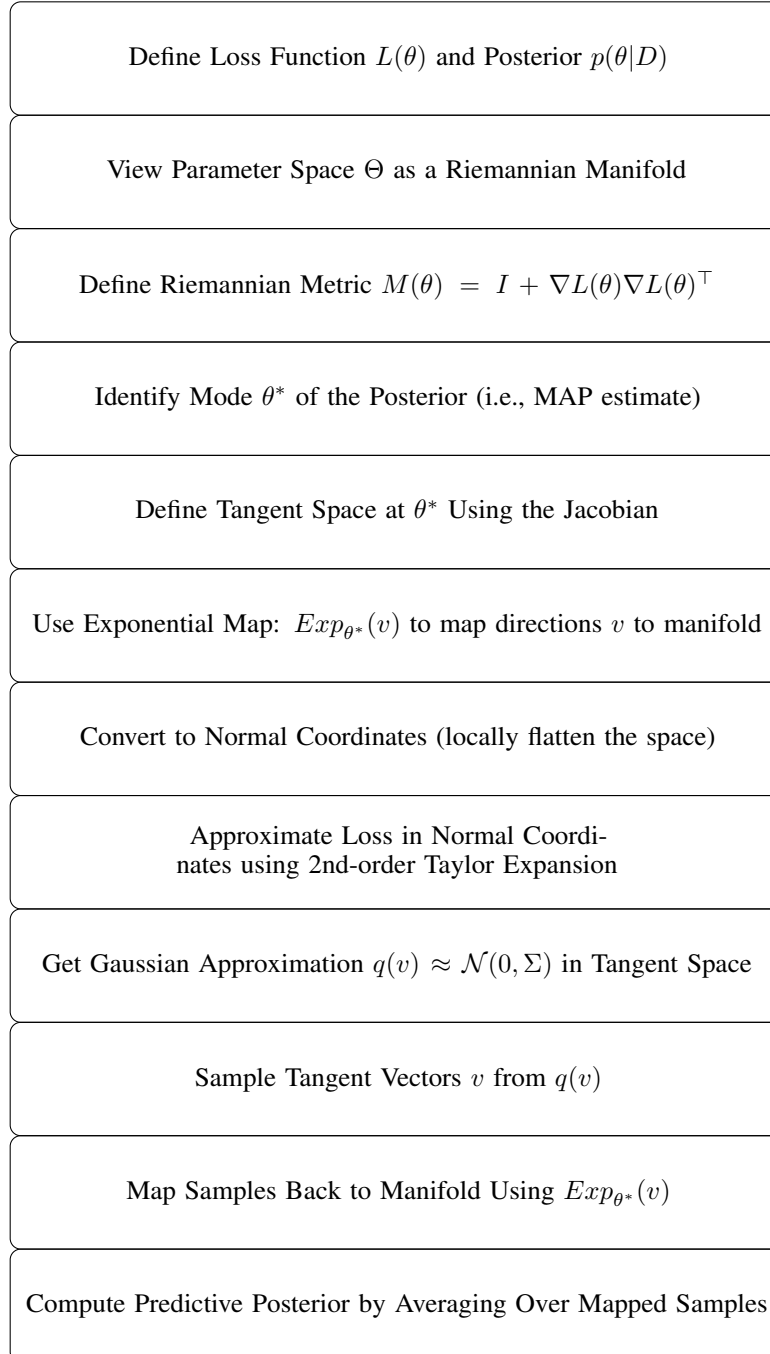
$$p(y | x', D) = \int p(y | x', D, \text{Exp}_{\theta^*}(v)) q_T(v) dv$$

This integral is approximated via Monte Carlo:

$$p(y | x', D) \approx \frac{1}{S} \sum_{s=1}^S p(y | x', D, \text{Exp}_{\theta^*}(v_s)), \quad v_s \sim q_T(v)$$

Here, tangent vectors v_s are sampled from the Gaussian approximation $q_T(v)$, then mapped back to the manifold using the exponential map. This process traces geodesic curves that reflect the local geometry of the loss landscape. As a result, the Riemannian posterior better captures the structure of the model around the mode θ^* , which is typically distorted or ignored in standard Laplace approximations.

The Riemannian Laplace approximation improves the traditional Laplace method by taking the shape or "geometry" of the loss landscape into account. Instead of treating the parameter space as flat, it uses a flexible, data-driven way to measure distances, allowing for a more accurate local approximation of the posterior. By using tools like the exponential map and normal coordinates, it creates a Gaussian approximation that better fits the true structure of the model. This leads to better uncertainty estimates and predictions. This method is more complex compared to the Laplace approximation since it requires knowledge of advanced concepts from Riemannian geometry. The following flowchart explains the approach in simplified manner:-



2.2 Strengths of paper:-

Better approximation and Predictive Performance:- The main strength of this method is that it allows for a more accurate approximation of the posterior distribution, especially in cases where the parameter space is curved or the posterior is far from Gaussian. By considering the geometry of the parameter space, this method can capture the non-linearities in the model that are ignored by the traditional Laplace approximation. Results from the original paper as well as experiments conducted by us (description given in later section) demonstrate the better performance of Riemannian LA in posterior approximation.

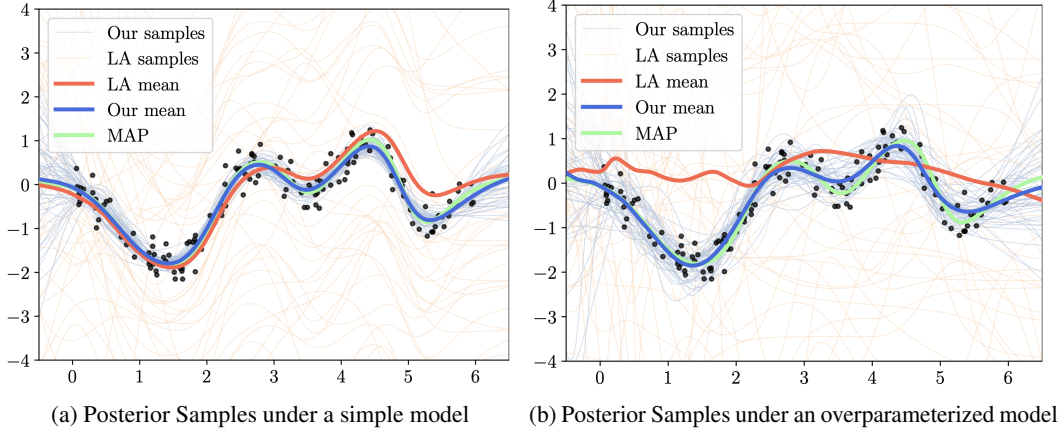


Figure 1: Figure from original paper showing usefulness of RLA

2.3 Weakness of paper:-

- **Code Inefficiencies:** The implementation provided by the authors for the classification task contained several issues, including runtime errors and non-functional components. Additionally, the sampling process was notably slow, indicating inefficiencies in the code design and execution.
- **High Computational Overhead:** A key limitation of the RLA approach lies in its high computational cost, particularly due to the need to integrate geodesic ordinary differential equations (ODEs). The cost of evaluating the ODE scales linearly with the number of observations, and in particular it is $O(SW N)$, where S is the number of steps of the solver, W is the number of model parameters and N is the dataset size. In the original implementation, this integration is performed using a CPU-based ODE solver, which is computationally expensive and suboptimal for large-scale or real-time applications. This high computational cost overhead of RLA can be clearly seen in the following figures.

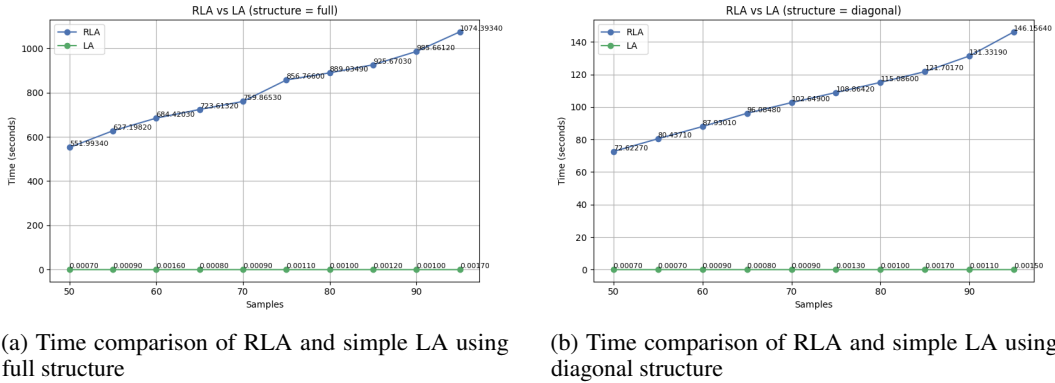


Figure 2: Comparison of runtime between RLA and LA under different structural assumptions.

Additionally, a detailed table containing the data for these plots can be found in the appendix

3 Our Work / Novelty :-

3.1 Testing RLA approach on diverse DATASETS:-

To assess the effectiveness of the Riemannian Laplace Approximation (RLA), we extend the original experiments by testing the approach on a broader range of datasets. While the original paper primarily focuses on binary classification and regression tasks, we additionally implement a multi-class classification model based on the RLA framework. This extended evaluation provides a deeper understanding of the RLA’s practical usability across different types of learning problems.

3.1.1 Experiments on Binary Classification

- **Concentric Circles Dataset** In this experiment, we compare the performance of the Laplace Approximation (LA) and the Riemannian Laplace Approximation (RLA) on the Concentric Circles (CC) dataset. The dataset consists of points arranged in concentric circles, commonly used to evaluate the performance of classifiers in non-linear classification tasks.

The plots below visualize the decision boundaries and classification accuracy of the different methods tested. The performance of each method is evaluated based on various metrics such as accuracy, negative log likelihood (NLL), Brier score, expected calibration error (ECE), and mean calibration error (MCE).

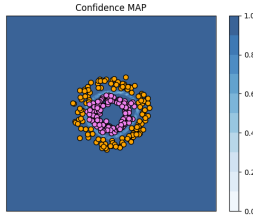


Figure 3: MAP Classification Result on Concentric Circles

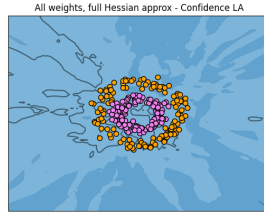


Figure 4: LA Classification Result on Concentric Circles

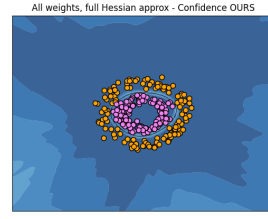


Figure 5: RLA Classification Result on Concentric Circles

Results Table

The table below summarizes the performance metrics for MAP, LA, and RLA on the Concentric Circles dataset:

Method	Accuracy (%)	NLL	Brier	ECE	MCE
MAP	98.97	0.01991	0.00632	1.04	58.35
LA	37.43	0.69662	0.25182	18.69	38.76
RLA	100.0	0.01604	0.00346	1.37	43.92

Performance Comparison for MAP, LA, and RLA on Concentric Circles

- **Custom Spiral Dataset Experiment**

The *custom spiral dataset* is a synthetic 2D classification dataset frequently used to evaluate model performance on non-linearly separable tasks. It contains two intertwined spirals representing different classes, with added Gaussian noise to simulate ambiguity. This setup challenges models to learn complex decision boundaries and serves as a benchmark for assessing uncertainty estimation and calibration.

Confidence Plots

To analyze model calibration, we visualize softmax confidence for each method:

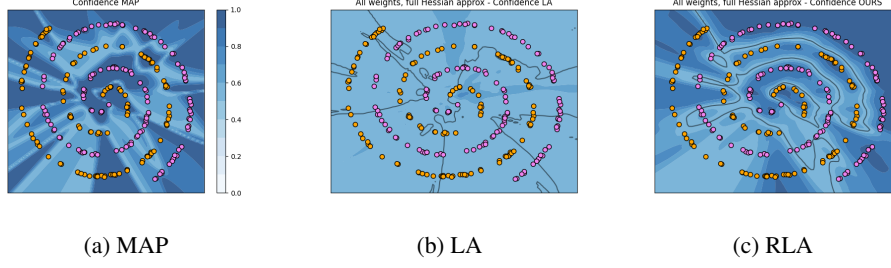


Figure 6: Confidence visualizations from different methods on the spiral dataset.

Accuracy and Calibration Metrics

Method	Accuracy	NLL	Brier	ECE	MCE
MAP	70.77%	0.5770	0.1888	5.85	9.82
LA	62.56%	0.6727	0.2399	7.99	17.02
OURS	84.10%	0.4333	0.1382	10.76	17.43

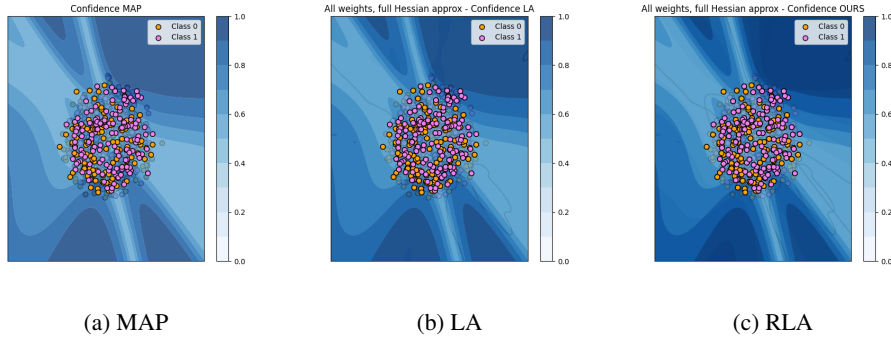
Comparison of accuracy and calibration metrics on the custom spiral dataset.

• Synthetic dataset

The *synthetic dataset* is generated using a polar coordinate transformation to form two complex spiral-like patterns for binary classification. The two classes are defined by phase-shifted sine curves, making the task non-trivial and suitable for evaluating model robustness and calibration.

The resulting data exhibits significant overlap and nonlinear separability, particularly challenging for standard classifiers.

Confidence Plots



Confidence plots for MAP, LA, and OURS methods on the synthetic dataset.

Accuracy and Calibration Metrics

Method	Accuracy	NLL	Brier	ECE	MCE
MAP	64.33%	0.6636	0.2356	5.25	18.64
LA	48.67%	0.6982	0.2525	6.03	18.01
RLA	63.33%	0.6633	0.2354	7.84	18.01

Accuracy and calibration metrics for Laplace approximation methods.

3.1.2 Extension to Multi-Class Classification

The original Riemannian Laplace Approximation (RLA) framework, as introduced in the paper, was applied to binary classification (e.g., Banana dataset) and regression problems. In this work, we extend the methodology to a significantly more complex and high-dimensional setting: multi-class classification, using the MNIST and FashionMNIST datasets. This required both architectural changes and adaptations in posterior sampling and manifold construction logic.

Code Adaptation and Dataset

We implemented a new pipeline in `multiclassification_experiment.py`, inspired by the existing binary classification framework. Unlike the binary case which used only a single feature, the MNIST dataset has $28 \times 28 = 784$ features per image and 10 output classes.

Key changes from the binary classification setup include:

- A new model architecture with two hidden layers (each with 256 neurons and Tanh activations) to accommodate the higher input dimensionality and class complexity.
- Use of the `CrossEntropyLoss` function to support multi-class labels.
- Evaluation metrics such as Accuracy, Negative Log Likelihood (NLL), Brier score, Expected Calibration Error (ECE), and Maximum Calibration Error (MCE) computed across 10 classes.
- Visualization tools including t-SNE projections and confidence histograms to better understand model behavior in high dimensions.

Results

MNIST Dataset:

```
python mnist_rla.py -structure diag -subset last_layer -samples 10
```

Method	Accuracy (%)	NLL	Brier	ECE	MCE
MAP	97.47	0.1003	0.0041	1.34	38.14
Laplace	97.45	0.0942	0.0041	0.77	70.84
Riemannian	97.45	0.0942	0.0041	0.77	70.82

Comparison of MAP, Laplace, and Riemannian Laplace methods on the MNIST dataset.

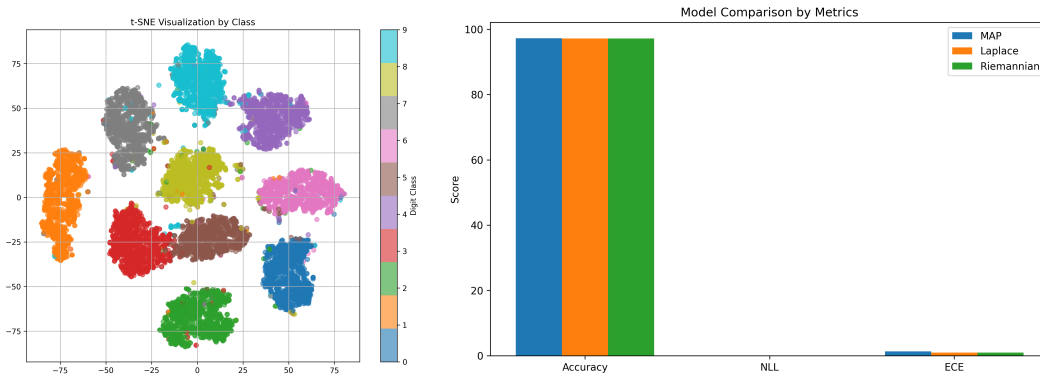


Figure 8: All three methods achieve comparable performance overall; however, RLA exhibits improved calibration, as reflected in lower ECE and MCE values compared to MAP and Laplace.

FashionMNIST Dataset:

```
python fashion_mnist_rla.py -structure diag -subset last_layer -samples 10
```

For this dataset, both Laplace Approximation (LA) and Riemannian Laplace Approximation (RLA)

underperformed compared to MAP. RLA performed slightly worse than LA in terms of uncertainty calibration and accuracy.

Method	Accuracy (%)	NLL	Brier	ECE	MCE
MAP	88.70	0.4173	0.0176	5.53	71.77
Laplace	62.70	1.3965	0.0574	16.74	29.91
Riemannian	9.04	3.5392	0.0977	20.61	55.65

Comparison of MAP, Laplace, and Riemannian Laplace methods on the FashionMNIST dataset.

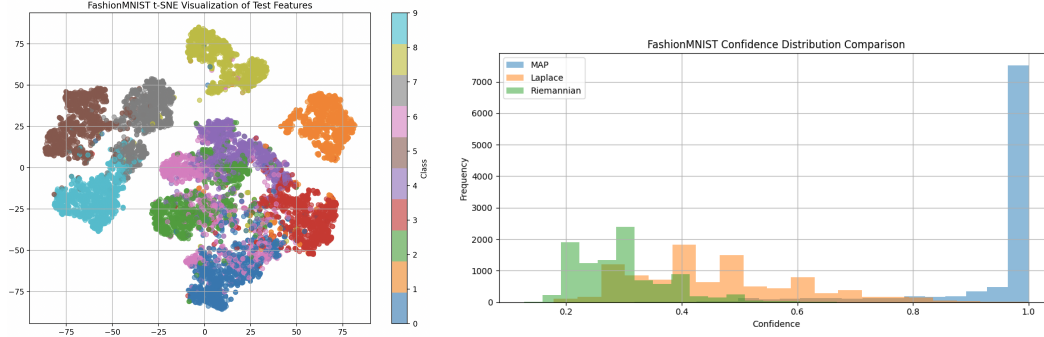


Figure 9: MAP significantly outperformed Laplace, which in turn showed notably better performance than RLA. In this case, RLA performed poorly, with an accuracy close to random chance (i.e., below 10% for a 10-class problem).

Inference

The experimental results reveal important observations:

- On MNIST, RLA showed marginal improvements in calibration (ECE) over MAP and similar performance to LA, indicating its ability to better model uncertainty in well-structured datasets.
- However, on FashionMNIST, RLA underperformed severely. This may be due to the dataset's higher intra-class variability and noisier features, making the geodesic manifold assumption less appropriate or harder to optimize over.
- The manifold structure constructed for RLA relies on curvature information that may not generalize well when feature representations are less separable or when the posterior landscape is less smooth.
- MAP-based solutions, while point-estimates, can outperform approximate Bayesian methods in noisy settings due to over-regularization or instability in approximating posterior curvature.

3.2 Removing errors and making code faster:-

- **Removing errors:-** The classifier code provided by authors had following major errors:-
 - ValueError: Expected primal to be floating point or complex, but got: torch.int64. Root cause-Root Cause: The jvp function in torch.func requires inputs (primals and tangents) to be of floating point or complex dtype. The previous code did not enforce this, resulting in a runtime error when integer tensors were passed.
 - Mismatch in Input Structure Passed to jvp:- The second argument to jvp must have the same structure as the first. Here, `params_map` is a nested tuple/dictionary-like structure, while `diff_weights` was a flat list. The mismatch in structure caused a failure in the forward-mode autodiff system.

- **Fixes and Refactoring:** The above issues were resolved by enforcing proper data types and aligning parameter structures. These changes were implemented in `manifold.py` and `functorch_utils.py`. The corrected versions of these files can be found in our project repository linked at the end of this report.
- **Speeding up the sample generation process:** To accelerate posterior weight sampling in our Riemannian Laplace Approximation experiments, we parallelized the computation of individual weight samples using the `joblib.Parallel` backend with `n_jobs=8`. This modification significantly reduced the total runtime of the weight sampling loop. The following table presents a comparison between the unoptimized and optimized execution times (in seconds) for the `time_weights_for_RLA` metric across different sample sizes (`samp`) and structural configurations (`full` and `diag`). The optimized results include additional metrics such as `time_weights_optimized_rla` and `time_weights_LA`. The speedup in code can be better visualised using following plots:-

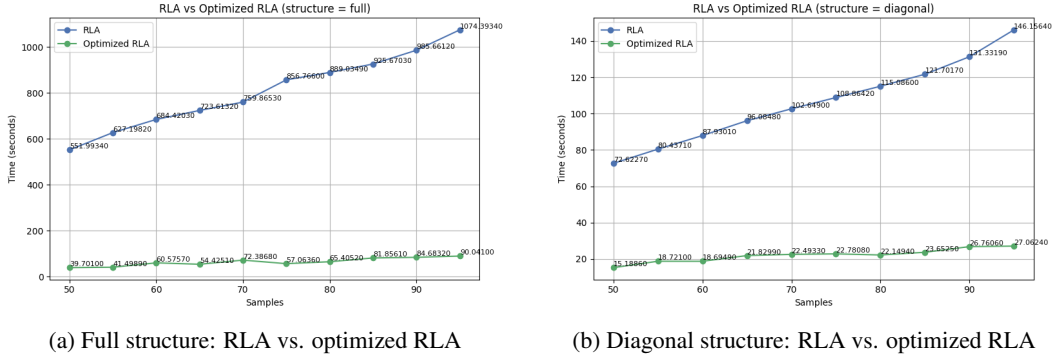


Figure 10: Runtime comparison of RLA and optimized RLA under full and diagonal structures.

Additionally, a detailed table containing the data for these plots can be found in the appendix

3.3 ODE Solving Improvement Approaches

In the Riemannian Laplace Approximation framework, the exponential map is used to sample from a posterior distribution defined on a manifold. This operation requires numerically solving a second-order ordinary differential equation (ODE). The following sections describe a series of progressive improvements designed to enhance the efficiency, reliability, and hardware utilization of this ODE solver.

3.3.1 Original ODE Solver (Baseline)

Algorithm 1 Original SciPy `solve_ivp` ODE Solver

```

function NEW_SOLVE_EXPMAP(manifold, x, v, ode_fun, subset_of_weights)
  Convert x and v to NumPy if needed
  init ← concatenate(x, v)
  solution ← solve_ivp(ode_fun, [0, 1], init, dense_output=True, atol=1e-3, rtol=1e-6)
  curve ← lambda tt: evaluate_solution(solution, tt, 1)
  return curve, False
end function

```

This baseline approach utilizes `scipy.integrate.solve_ivp` to numerically solve the geodesic ODE. While this implementation is correct, it relies entirely on CPU computation and requires multiple conversions between NumPy and PyTorch tensors, which introduces additional overhead. The use of dense output further increases memory usage and runtime.

3.3.2 SciPy's odeint Solver

Algorithm 2 SciPy odeint Solver

```
function NEW_SOLVE_EXPMAP(manifold, x, v, ode_fun, subset_of_weights)
    Convert inputs to NumPy
    init ← concatenate(x, v)
    t_eval ← linspace(0, 1, 10)
    solution ← odeint(ode_fun, init, t_eval, tfirst=True)
    Create cubic interp1d function from solution
    return lambda tt: interpolated positions and velocities, False
end function
```

This variation replaces `solve_ivp` with `scipy.odeint`, which avoids some dense output overhead and uses internal error control strategies better suited for certain smooth problems. A cubic interpolation strategy is employed for efficient intermediate evaluation.

3.3.3 TorchDiffEq GPU Solver

Algorithm 3 TorchDiffEq GPU Solver

```
function NEW_SOLVE_EXPMAP(manifold, x, v, ode_fun, subset_of_weights)
    x, v ← convert to Torch tensors on same device
    init ← concatenate(x, v)
    t_eval ← linspace(0, 1, 10)
    torch_ode_fun ← second2first_order(manifold, y, subset_of_weights)
    solution ← torch_odeint(torch_ode_fun, init, t_eval)
    return lambda tt: slices of solution, False
end function
```

In this implementation, the ODE solver is fully transitioned to the GPU using the `torchdiffeq` library, ensuring that all computations are performed within the PyTorch framework. This eliminates dependencies on CPU-based operations (such as NumPy), thereby removing costly device transfers and significantly accelerating execution—particularly for high-dimensional deep neural networks—as emphasized in the original research.

3.3.4 Hybrid TorchDiffEq Approach

Algorithm 4 Hybrid TorchDiffEq Solver

```
function NEW_SOLVE_EXPMAP(manifold, x, v, ode_fun, subset_of_weights)
    x, v ← torch tensors on device
    t_eval ← linspace(0, 1, 5)
    solution ← torch_odeint(..., method='dopri5')
    return interpolated curves using interp, False
end function
```

This version balances memory usage and compute speed by solving with fewer time steps and using tailored linear or cubic interpolation for intermediate values. This is particularly useful when geodesics must be computed in parallel for a batch of samples.

3.3.5 Production-Grade Solver

Algorithm 5 Production-Grade Solver

```
function NEW_SOLVE_EXPMAP(manifold, x, v, ode_fun, subset_of_weights)
    Convert x, v with safe dtype/device handling
    try:
        Solve with torch_odeint and adaptive tolerances
        Interpolate and return curve
    except:
        Return fallback constant curve
end function
```

This robust version includes adaptive tolerance control and fallback mechanisms. If the solver fails or returns invalid outputs (e.g., NaNs), a default constant-speed geodesic is returned. This makes the system more reliable for deployment.

3.3.6 Numba-Accelerated Solver (Untested)

Algorithm 6 Numba-Based ODE Solver

```
function NEW_SOLVE_EXPMAP(manifold, x, v, ode_fun, subset_of_weights)
    init ← concatenate(x, v)
    manifold_params ← extract static params
    solution ← solve_ivp with numba_ode_fun
    Interpolate if needed
    return curve, False
end function
```

Although not tested in this work, an additional path involves implementing the ODE function in low-level code using numba. This may improve performance on CPU-only systems with small to medium model sizes, especially for embedded or real-time applications.

3.3.7 Comparison Summary

Version	Improvement	Compatible	Best For
Original	Baseline	Yes	Reference only
odeint	CPU Speed	Yes	Simple CPU jobs
TorchDiffEq	GPU Use	Partial	High-D DNNs
Numba	CPU Max Speed	No	Embedded/Static
Hybrid	Balanced	Yes	General Use
Production	Robustness	Yes	Critical Apps

4 Future Work

Towards the conclusion of this project, while exploring ways to enhance the performance of our ODE solver, we encountered the concept of Neural ODEs [2]. These models represent a class of deep learning architectures that integrate differential equation solvers as black-box components. Although promising, we were unable to implement this approach due to time constraints and limited familiarity with the underlying techniques.

As part of future work, we plan to integrate Neural ODEs into the Riemannian Laplace Approximation (RLA) framework to investigate their effectiveness in modeling the geodesic flows required for posterior sampling.

Additionally, our experiments applying RLA to multi-class classification tasks—particularly on datasets like FashionMNIST—revealed cases where RLA underperformed relative to the standard Laplace Approximation (LA). A deeper, systematic comparison between RLA and LA across varying dataset types and model architectures remains an important direction for future exploration.

5 Learnings from the Project

A significant portion of the project was dedicated to understanding the principles of Riemannian Geometry and its role in approximating intractable posterior distributions through the Riemannian Laplace Approximation (RLA). Additionally, designing a GPU-compatible ODE solver and optimizing it for performance was a key technical milestone. The following are the key takeaways from the project:

- **Hardware Utilization Matters:** Migrating to GPU-based solvers like `torchdiffeq` significantly improved performance, especially for high-dimensional models.
- **Avoid Data Transfer Overhead:** Eliminating NumPy-PyTorch conversions reduced memory overhead and improved runtime by keeping computations entirely in PyTorch.
- **Interpolation Strategy is Crucial:** Using fewer time steps with efficient interpolation (e.g., cubic or clamped linear) balanced accuracy and memory usage effectively.
- **ODE Tolerances Need Careful Tuning:** Fine-tuning `atol` and `rtol` impacted both solver reliability and efficiency; incorrect values led to either slowness or instability.
- **RLA as a Posterior Approximation Tool:** We learned how RLA provides a geometric approach to posterior sampling by leveraging the curvature of the loss landscape. It extends classical Laplace approximation to manifolds, allowing for richer uncertainty estimation.
- **Parallelism Accelerates Sampling:** We experimented with parallelizing posterior sampling (e.g., solving multiple exponential maps simultaneously), which can yield close to linear speedups proportional to the number of available CPU/GPU cores.
- **ODE Solver Bottlenecks in ML Pipelines:** We discovered that inefficient ODE solvers can become a major bottleneck in RLA. Optimizing them—both numerically and at the hardware level—can drastically speed up posterior approximation and uncertainty quantification.
- **Dataset Characteristics Affect RLA Performance:** Applying RLA to different datasets (e.g., MNIST vs. FashionMNIST) revealed its sensitivity to the underlying data distribution and model architecture. While RLA worked well for structured, low-noise data like MNIST, it struggled with more complex, noisy datasets like FashionMNIST.

6 Code Repository

The complete implementation and experiments described in this report are available at:

<https://github.com/PKING1501/riemannian-laplace-approximation>

7 Contributions

All three members contributed equally throughout the project. We jointly developed the theoretical understanding of Riemannian Laplace Approximation (RLA), including its formulation, manifold-based sampling, and posterior approximation.

Implementation tasks—such as extending RLA to multi-class classification, experimenting with different ODE solvers, and applying the method to various datasets—were divided fairly and executed collaboratively. We actively supported each other during debugging, optimization, and experimentation.

References

- [1] Federico Bergamin, Pablo Moreno-Muñoz, Søren Hauberg, and Georgios Arvanitidis. Riemannian laplace approximations for bayesian neural networks, 2023.
- [2] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations, 2019.
- [3] Levent Sagun, Leon Bottou, and Yann LeCun. Eigenvalues of the hessian in deep learning: Singularity and beyond, 2017.

Appendix

A Comparison of LA and Simple RLA latency

As discussed in earlier sections, we examined the differences between the traditional Laplace Approx. method and the simple RLA approach through plots. Here, we present a detailed table, Table ??, which compares the performance of both methods. The results show that while RLA typically incurs a higher computational overhead, it outperforms the LA method for larger sample sizes. These observations underscore the trade-offs between the two methods, particularly in terms of execution time and their suitability for different datasets

Table 1: Comparison of LA and RLA Time

samp	structure	RLA (seconds)	LA (seconds)
50	full	551.9934	0.0007
55	full	627.1982	0.0009
60	full	684.4203	0.0016
65	full	723.6132	0.0008
70	full	759.8653	0.0009
75	full	856.7660	0.0011
80	full	889.0349	0.0010
85	full	925.6703	0.0012
90	full	985.6612	0.0010
95	full	1074.3934	0.0017
50	diag	72.6227	0.0007
55	diag	80.4371	0.0007
60	diag	87.9301	0.0009
65	diag	96.0848	0.0008
70	diag	102.6490	0.0009
75	diag	108.8642	0.0013
80	diag	115.0860	0.0010
85	diag	121.7017	0.0017
90	diag	131.3319	0.0011
95	diag	146.1564	0.0015

B Comparison of RLA and Optimized RLA

As mentioned in the previous sections, we discussed the optimizations and presented plots comparing the original RLA approach with the optimized RLA method. Here, we provide a detailed table, Table 2, which highlights the runtime performance of both methods when applied to full and diagonal structures. The table further confirms that the optimized RLA approach results in a substantial reduction in execution time, with the improvements being especially evident for larger sample sizes.

Table 2: Comparison of RLA Time (Optimized vs. Non-Optimized)

samp	structure	Unoptimized RLA (seconds)	Optimized RLA (seconds)
50	full	551.9934	39.7010
55	full	627.1982	41.4989
60	full	684.4203	60.5757
65	full	723.6132	54.4251
70	full	759.8653	72.3868
75	full	856.7660	57.0636
80	full	889.0349	65.4052
85	full	925.6703	81.8561
90	full	985.6612	84.6832
95	full	1074.3934	90.0410
50	diag	72.6227	15.1886
55	diag	80.4371	18.7210
60	diag	87.9301	18.6949
65	diag	96.0848	21.8299
70	diag	102.6490	22.4933
75	diag	108.8642	22.7808
80	diag	115.0860	22.1494
85	diag	121.7017	23.6525
90	diag	131.3319	26.7606
95	diag	146.1564	27.0624